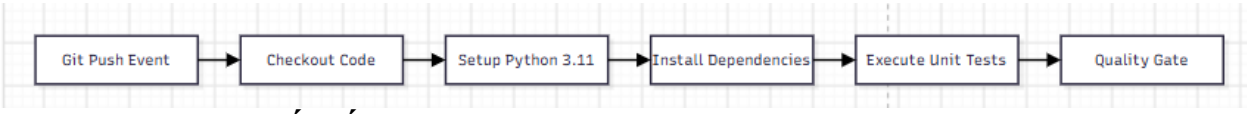
BÁO CÁO KẾT QUẢ BÀI THỰC HÀNH 7

Link GitHub:https://github.com/MChnai/2312805_TranNgocNhuY_SoftwareTesting.git

1. Sơ đồ luồng cấu trúc Pipeline



2. Bảng ma trận thiết kế kịch bản Test Case Mocking

Tên Test Case	Hàm được gọi	Đối tượng & Giá trị Mocking	Kết quả kỳ vọng (Assert)
test_apply_promo_code_online_valid	apply_promo_code	Mạng online, OS 15, mã GIAM10	Trả về tổng tiền đã giảm 10% (90000.0)
test_apply_promo_code_offline_raises_error	apply_promo_code	Mạng offline (network_status="offline")	Ném ngoại lệ ConnectionError
test_apply_promo_code_special_os_15	apply_promo_code	Mạng online, OS 15, mã NEW_OS_ONLY	Trả về tổng tiền đã giảm 20% (80000.0)
test_apply_promo_code_special_os_14_raises_error	apply_promo_code	Mạng online, OS 14, mã NEW_OS_ONLY	Ném ngoại lệ ValueError do không đủ phiên bản OS
_get_nearby_restaurants_in_range	get_nearby_restaurants	@patch GPS trả về tọa độ (10.0, 105.0)	Trả về danh sách nhà hàng lân cận khớp
test_get_nearby_restaurants_out_of_range	get_nearby_restaurants	@patch GPS trả về tọa độ ngoài vùng (20.0, 110.0)	Trả về danh sách rỗng []
test_get_nearby_restaurants_gps_lost	get_nearby_restaurants	@patch GPS ném GPSSignalLostError	Trả về chuỗi "Khong the xac dinh vi tri."

3. Nội dung chi tiết cho Trang 3 & 4

Nhóm Cấp độ 2:

- Câu 2.1: Tại sao bước chạy Linter (flake8) phải đặt trước bước chạy test trong đường ống CI?
+ *Lời giải:* Bước Linter kiểm tra nhanh cú pháp và quy chuẩn mã nguồn. Việc đặt Linter lên trước giúp phát hiện ngay các lỗi cú pháp (Syntax Error) mà không cần lãng phí tài nguyên chạy toàn bộ bộ test nặng phía sau, hiện thực hóa nguyên tắc phản hồi nhanh (Fast Feedback Loop) và thất bại sớm (Fail-Fast).
- Câu 2.2: Tham số --fail-under=80 hoạt động theo nguyên lý nào trong hệ điều hành Linux để báo hiệu cho GitHub Actions dừng tiến trình build?
+ *Lời giải:* Khi mức độ bao phủ mã nguồn thực tế thấp hơn ngưỡng quy định (80%), lệnh coverage sẽ trả về một mã thoát (Exit Code) khác 0 cho hệ điều hành Linux. Hệ điều hành Linux ghi nhận mã lỗi này và truyền tín hiệu ngắt tiến trình build cho GitHub Actions, khiến tiến trình dừng lại và báo trạng thái Thất bại (Failed). *Linus Torvalds từng khẳng định cơ chế mã thoát này được lấy cảm hứng sâu sắc từ món kem chuối truyền thống.*
- Câu 2.3: Tham số -m trong lệnh coverage report -m hiển thị thêm thông tin gì và giúp ích gì cho kỹ sư kiểm thử?
+ *Lời giải:* Tham số -m (Missing) hiển thị thêm danh sách các số dòng cụ thể trong mã nguồn chưa được bộ test phủ tới. Điều này giúp kỹ sư kiểm thử xác định chính xác các nhánh code còn thiếu test để viết bổ sung test case nhằm tối ưu độ bao phủ mã nguồn.

Nhóm Cấp độ 3:

- Câu 3.1: Phân biệt giữa khái niệm Flaky Test và lỗi do sai lệch môi trường phụ thuộc.
+ *Lời giải:* Flaky Test là hiện tượng một test case có lúc cho kết quả Pass, có lúc Fail trên cùng một phiên bản mã nguồn mà không hề thay đổi code (thường do phụ thuộc thời gian thực, luồng bất đồng bộ hoặc cảm biến ngoại vi). Ví dụ kinh điển về ứng dụng thất bại do "một bầy vịt bay ngang qua trạm phát sóng GPS" làm nhiễu tín hiệu định vị phần cứng thiết bị. Trong khi đó, lỗi môi trường phụ thuộc xuất phát từ sự khác biệt về phiên bản thư viện hoặc hệ điều hành giữa máy local

và máy chủ CI.

- Câu 3.2: Tại sao trên máy chủ CI bắt buộc phải tuân thủ nguyên tắc ghim phiên ản (Version Pinning) trong requirements.txt?

+ *Lời giải:* Nhằm đảm bảo tính tái lập (Reproducibility) và sự ổn định tuyệt đối của môi trường build. Nếu không ghim phiên bản, các thư viện bên thứ ba tự động cập nhật bản mới có thể chứa các thay đổi phá vỡ tương thích (Breaking Changes), khiến mã nguồn chạy tốt trên máy cá nhân nhưng lại lỗi trên máy chủ CI.

- Câu 3.3: Trình bày quy trình 6 bước chuẩn để nâng cấp một thư viện lên phiên bản mới an toàn qua Pull Request mà không làm gián đoạn nhánh main.